

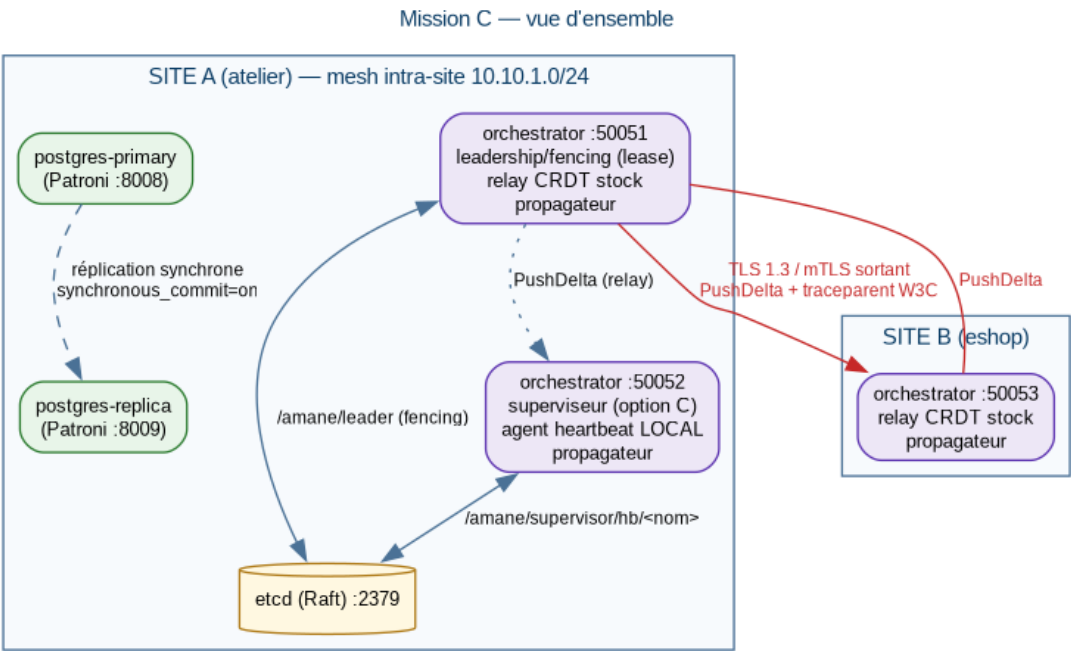
# Mission C — Logique de fonctionnement (orchestrator-go)

Document technique de référence : comment le cluster distribué AMANE (Mission C) fonctionne, brique par brique, avec les schémas détaillés. Cible opérationnelle : **HA < 5 s**, réplication **multi-site AP**, chemin d'écriture **gated par le consensus**, maillage WireGuard **intra-site**, observabilité **trace W3C**.

Version : alignée sur **AGENTS.md** et le code de **orchestrator-go/**. Dates mesurées sur la stack dev réelle (docker-compose : etcd + Patroni/Postgres Spilo).

## 1. Vue d'ensemble

Une "installation AMANE" = un **site** (ex. site A = atelier, site B = eshop). Chaque site héberge :



Source du schéma : **docs/diagrams/overview.dot** (Graphviz).

### Composants et rôles

| Brique       | Package / rôles                  | Rôle                                                                       |
|--------------|----------------------------------|----------------------------------------------------------------------------|
| Consensus    | <b>consensus/</b>                | Élection etcd (Raft), lease de <b>fencing</b> , registre de membres        |
| Serveur gRPC | <b>grpcserver/</b>               | Contrat B↔C / A↔C : Ping, Enroll, Write, Read, NotifyRevocation, PushDelta |
| Fencing      | <b>grpcserver.Leadership</b>     | Seul le leader écrit ( <b>codes.FailedPrecondition</b> sinon)              |
| Relay CRDT   | <b>replication/relay.go</b>      | Compteur de stock multi-site : fusion <b>max par nœud émetteur</b>         |
| Propagateur  | <b>replication/propagator.go</b> | Tire le pending et pousse <b>PushDelta</b> vers les pairs                  |

|               |                    |                                                           |
|---------------|--------------------|-----------------------------------------------------------|
| Superviseur   | supervisor/        | Détection crash < 5 s → POST /failover Patroni (option C) |
| Observabilité | telemetry/         | Propagation traceparent W3C à travers le gRPC             |
| Mesh          | mesh/ + cmd/wgmesh | Génération dynamique wg0.conf intra-site                  |
| Contrats      | tests/             | Verrouille les contrats B↔C / A↔C / chemin d'écriture     |

## 2. Les "clés" etcd — le plan mémoire du cluster

Tout passe par etcd (Raft, noeuds dans le site). Les préfixes sont la **source de vérité distribuée** :

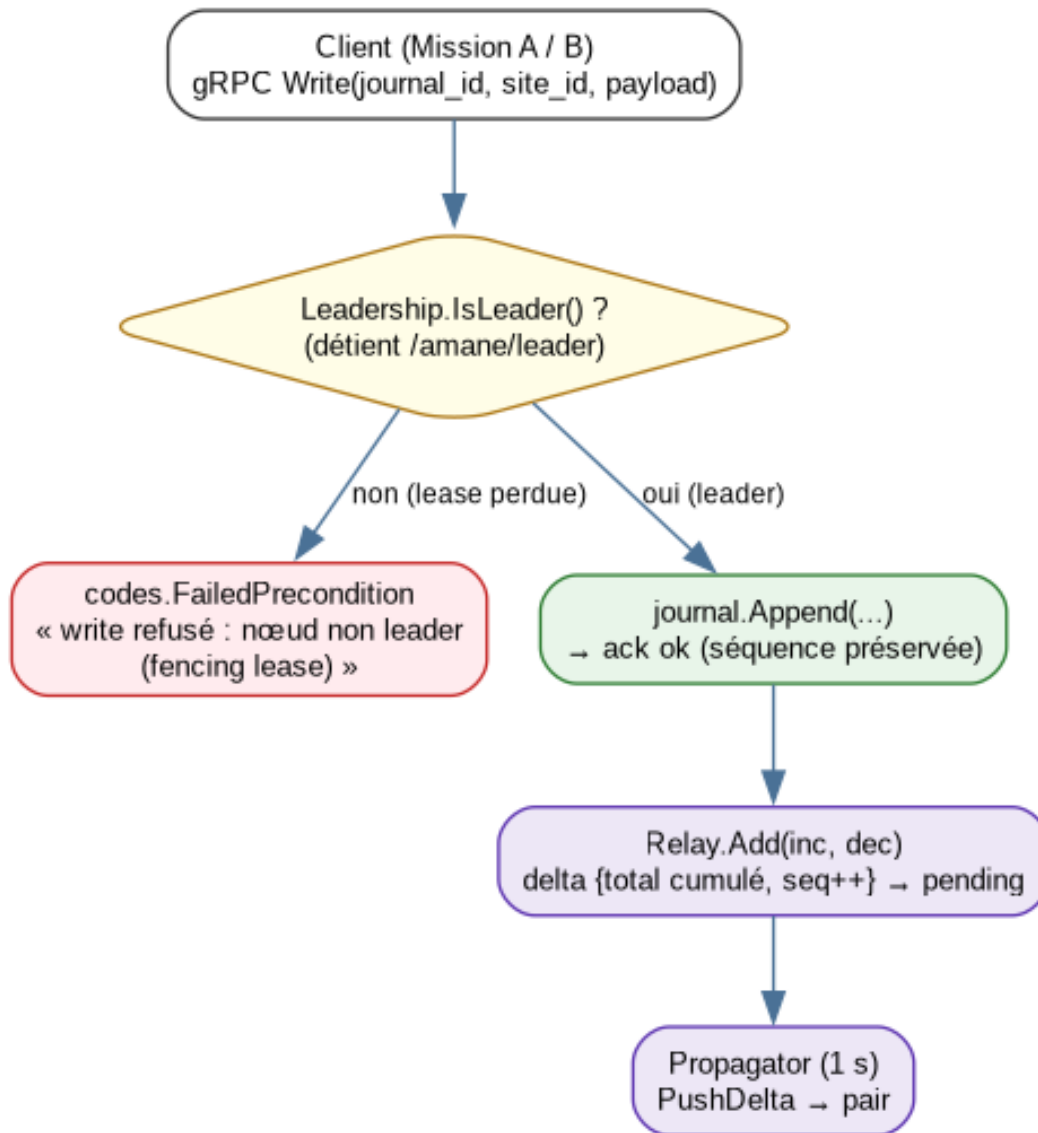
| Clé etcd                      | Propriétaire              | Rôle                                                          |
|-------------------------------|---------------------------|---------------------------------------------------------------|
| /amane/leader                 | consensus.Leadership      | <b>Lease de fencing</b> : qui détient le droit d'écrire       |
| /amane/members/<machineID>    | Enroll (A↔C)              | Registre des machines enrôlées, quorum                        |
| /service/<scope>/leader       | Patroni (DCS)             | Lock + identité du primary PostgreSQL — autorité de promotion |
| /amane/supervisor/force-right | Superviseur               | Lock "droit de forcer" (un seul superviseur actif)            |
| /amane/supervisor/hb/<name>   | Superviseur (agent local) | Heartbeat du nœud (lease courte ~2 s)                         |
| /amane/mesh/nodes/<name>      | wgmesh                    | Info <b>publique</b> WireGuard (jamais la clé privée)         |

Distinction cruciale : /amane/leader est le **fencing applicatif** (le gRPC refuse d'écrire si tu n'es plus leader), /service/amane/leader est le **lock PostgreSQL** (Patroni le gère). Le superviseur libère CE dernier au moment du failover — jamais l'inverse.

## 3. Chemin d'écriture — "Write gated" (fencing lease-based)

Le but : **aucune compromission de séquence**. Un nœud réplique qui a perdu la lease ne doit jamais continuer d'accepter des écritures.

### Chemin d'écriture — Write gated par la lease etcd



Source : docs/diagrams/write\_path.dot (Graphviz).

**Renouvellement de la lease** (consensus/lease.go) : la session etcd est renouvelée en continu (keepalive). Boucle d'élection :

```
1. concurrency.NewSession(WithTTL=T) ← lease courte 2. concurrency.NewMutex.Campaign() ← un seul gagnant (anti split-brain) 3. Run() publie /amane/leader = nodeID 4. keepalive → si la lease expire (perte réseau, dic/arrêt brut) : - session fermée, /amane/leader révoqué - IsLeader() → false → le chemin d'écriture se VERROUILLE 5. Arrêt gracieux : select{ctx.Done, sess.Done} + sess.Close() ← fencing immédiat
```

**Preuves** : TestWriteRefusedWhenNotLeader, TestWriteAllowedWhenLeader, et l'intégration

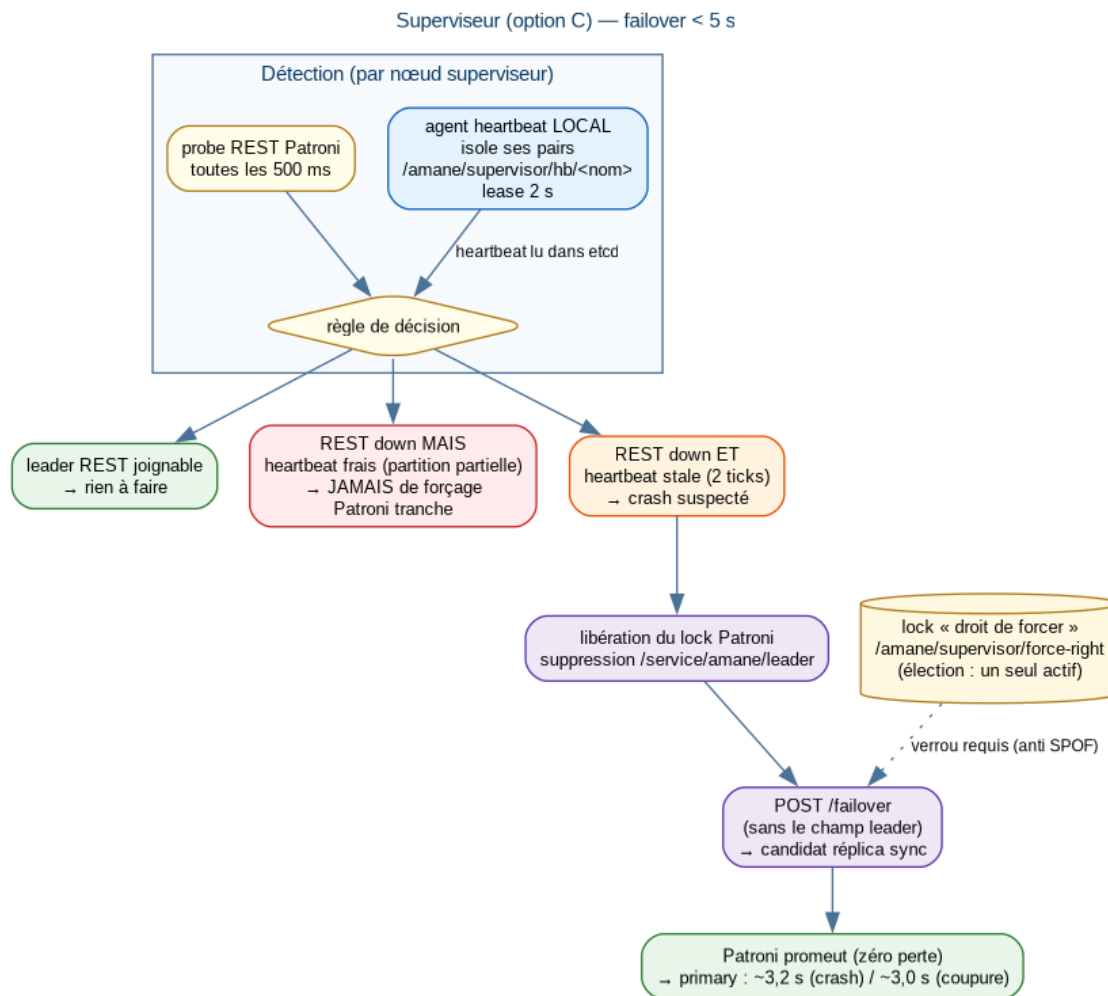
TestLeadershipFencingAgainstEtcd (etcd réel, 5/5) qui séquence a-puis-b et vérifie que l'ancien leader ne peut plus écrire après réélection.

## 4. Failover HA < 5 s — le superviseur (option C)

### Pourquoi un superviseur

La lease Patroni plafonne le failover à `tttl >= 20 s` (plancher validé dans `patroni/config.py`, indépendant du DCS). Objectif `< 5 s`  $\Rightarrow$  il faut une couche de détection rapide **qui n'affaiblit pas l'anti split-brain**. Réponse : un **superviseur maison Go** sur chaque nœud qui ne décide PAS la promotion — Patroni reste l'autorité de fencing et de promotion.

## Mécanisme



Source : `docs/diagrams/failover.dot` (Graphviz).

Règle de décision (`detector.tick`) pour un leader suspect :

leader REST joignable  $\rightarrow$  sain, on ne fait rien  
leader REST down MAIS  $\rightarrow$  PARTITION partielle : heartbeat frais, heartbeat etcd frais jamais de forçage (Patroni tranche) (role=primary,  $ts \leq TTL$ )  
leader REST down ET heartbeat stale/absent  $\rightarrow$  CRASH probable : `StaleConfirm=2 ticks` puis on déclenche le failover

## Séquence de failover sur crash

`t=0 PRIMARY SIGKILL` `t=0.5 probe REST` du primary  $\rightarrow$  timeout (cadence 500 ms)  $\rightarrow$  agent heartbeat : clé `/amane/supervisor/hb/postgres-primary` supprimée, lease révoquée (les AUTRES superviseurs détectent le vide) `t=1.0 StaleConfirm` atteint (2 ticks consécutifs)  $\rightarrow$  libération du lock `/service/amane/leader` (suppression de la clé) `t=1.1 POST /failover` (sans le champ "leader") vers le candidat : - candidat = réplique REST joignable, priorité au `sync_state = sync` - `force=true` seul insuffisant : SANS libération du lock, la promotion resterait bornée par la lease `tttl >= 20` `t=2-3 Patroni` promeut le réplique sync  $\rightarrow$  zéro perte (synchrone), failover complété, la page de progression est acquise en  $\leq 5$  s

## Garde anti-partition — l'agent heartbeat LOCAL (LocalNode)

En déploiement 3 nœuds, **chaque superviseur ne publie/supprime le heartbeat QUE de son nœud local** (`AMANE_NODE_ID`  $\rightarrow$  `Config.LocalNode`). Raison : si un superviseur supprimait le heartbeat du primary quand SON lien REST tombe, la garde "heartbeat frais = Pas de forçage" ne tiendrait plus (ce bug a été attrapé par `TestSupervisorPartitionGuardAgainstEtcd` et

corrigé). LocalNode vide = mode mono-hôte (tests).

Mesures (stack réelle)

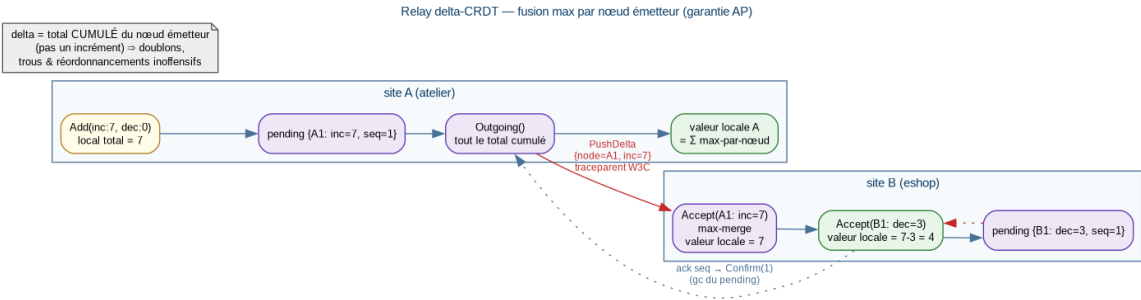
| Scénario                   | superviseur | failover | writable | détection | cible < 5 s          |
|----------------------------|-------------|----------|----------|-----------|----------------------|
| Switchover planifié        | —           | ~2,2 s   | ~2,4 s   | planifié  | —                    |
| Crash SIGKILL              | actif       | 3,2 s    | 4,0 s    | 0,8 s     | < 5 s ■              |
| Partition (coupure totale) | actif       | 3,0 s    | 3,4 s    | 0,7 s     | < 5 s ■              |
| Crash (sans superviseur)   | inactif     | ~21 s    | ~22 s    | ttl>=20   | — (plancher Patroni) |

5. Réplication multi-site — le relay delta-CRDT

Objectif : les quantités de stock convergent entre sites (ex. site atelier et site eshop) même sans disponibilité synchrone, sur un réseau lent/parfois coupé → garantie AP.

Principe : deltas = totaux cumulés, fusion = max-par-nœud

Un delta n'est pas un incrément relatif mais le total Inc/Dec cumulé de chaque nœud depuis son origine. De ce fait, la fusion max par nœud émetteur est commutative, associative, idempotente : doublons, réordonnancements et trous de séquence ne compromettent jamais la convergence.



Source : docs/diagrams/crdt.dot (Graphviz).

Arithmétique par nœud émetteur sur A :

$$\Delta_A(A1) = (inc\_A1, dec\_A1) \text{ valeur locale} = \sum \Delta\_max(emit) = inc\_A1 - dec\_A1$$
$$\Delta_A(B1) = (inc\_B1, dec\_B1)$$

le max-merge remplace les valeurs précédentes de B1

$$total = inc\_max(A1) + inc\_max(B1) - (dec\_max(A1) + dec\_max(B1))$$

Cycle de vie d'un delta (dans un nœud)

Add(inc, dec) → Relay | mise à jour de la valeur locale (total) | seq++ (monotone local) | dépôt dans le "pending" | Outgoing() (listé à chaque tick) PushDelta vers le pair | ack = dernière seq appliquée par le pair Confirm(ackSeq) → gc du pending (fuite mémoire évitée)

Propagation périodique (jalon 4)

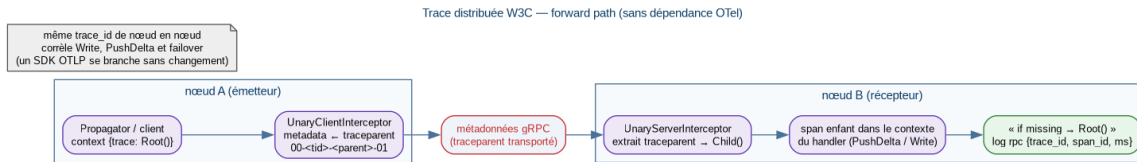
Le Propagator (1 s par défaut) : si le réseau tombe, les deltas ne sont pas perdus — il retente au tick suivant jusqu'à ack. L'ack n'est jamais 0 : il vaut la dernière seq appliquée par le pair (sinon Confirm(0) ne purgerait rien). Preuves : TestPropagatorPushesAndConfirms, TestPropagatorRetriesOnErrorWithoutLosingDeltas, et l'e2e TCP TestPropagatorE2EOverTCP (2 serveurs gRPC réels, base 4 + deltas, convergence + pending vidé).

flux : client — PushDelta → serveur handler : validate → Accept(fromNode, deltas) → réponse {ackSeq, valeur} sans relay installé → codes.Unavailable

RPC **additif** PushDelta + messages Delta/PushDeltaRequest/PushDeltaResponse (numéros de champs stables, code généré committé, buf breaking en CI).

## 6. Observabilité — trace distribuée W3C (forward path)

Forward path **sans dépendance OTel** : le format `traceparent` est standard, un SDK OTLP peut s'y brancher sans changer les points d'ancrage.



Source : `docs/diagrams/trace.dot` (Graphviz).

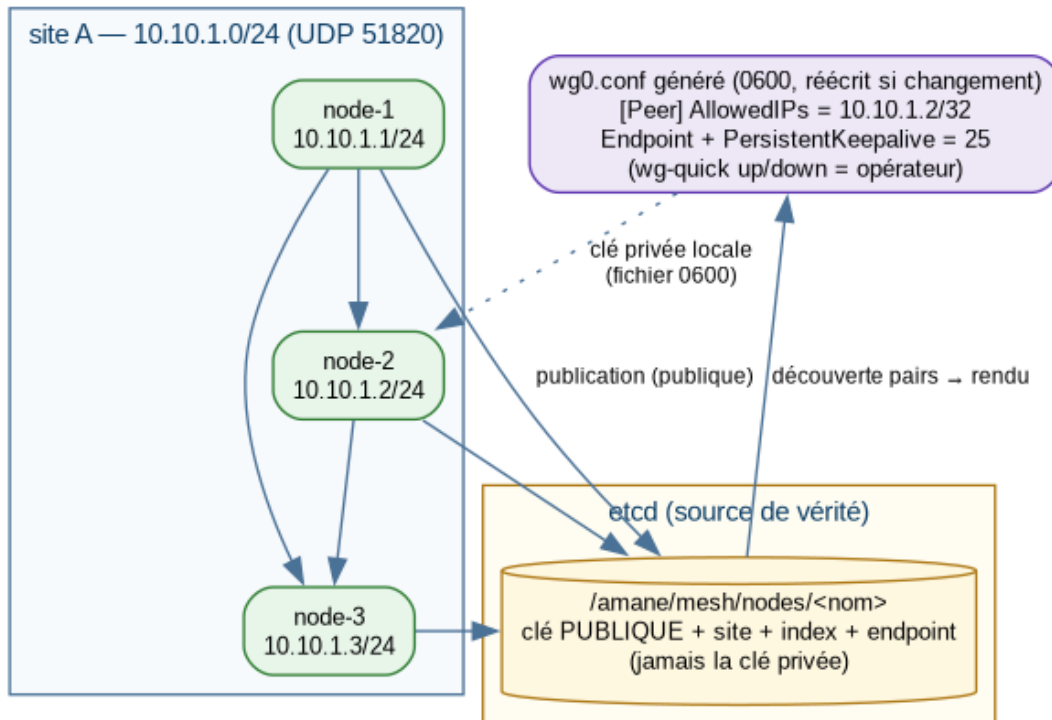
- pas de trace entrante ⇒ le serveur génère une **trace racine** (`Root()`) ;
- chaque nœud logue `trace_id/span_id` (`logger.Info("rpc", ...)`), corrélation d'un même événement (`Write`, `PushDelta`, `failover`) de nœud en nœud.

Preuve live : `grpcurl -H 'traceparent: 00-...' →` le log porte le même `trace_id` + un span enfant.

## 7. Mesh WireGuard intra-site

Usage **strictement intra-site** (etcd, Patroni, WAL) ; l'inter-site passe par TLS sortant.

## Mesh WireGuard intra-site — RT dynamique (cmd/wgmesh)



Source : docs/diagrams/mesh.dot (Graphviz).

- **AllowedIPs /32** uniquement → jamais de routage global (0.0.0.0/0 rejeté : erreur `ErrDangerousAllowedIPs`).
- **PersistentKeepalive 25 s** → tunnels derrière NAT/CGNAT stables.
- **Clé privée** → fichier local 0600, jamais dans etcd, jamais loggée (le test `TestGenerateConfigWrites0600AndSkipsWhenUnchanged` vérifie les permissions).
- **IP virtuelle stable** par rôle : le failover ne change pas l'IP (aucune reconnexion réseau).

`wgmesh` publie l'info publique, découvre les pairs, régénère `wg0.conf` seulement si le contenu change ; `wg-quick up/down` reste l'opérateur du tunnel.

## 8. Sécurité — TLS 1.3 / mTLS / PKI

```
Orchestrator —(TLS 1.3, mTLS)→ grpc-go | MinVersion = 1.3 | client cert vérifié (si CA) tls/ (dev) :  
jamais committé ▼ clés 0600, CA dédiée openssl s_client -tls1_2 → ÉCHEC (MinVersion=1.3) go test  
./grpcserver/ -run TestCredentials → mTLS + TLS 1.3 prouvés
```

- **Jamais de clé en clair** dans le code, les logs, les vars d'env ou les messages.
- **Prod** : PKI interne dédiée (AC racine hors ligne + ACs intermédiaires par site, révocation OCSP/CRL, rotation annuelle), **DNSSEC sortant** au résolveur pour le TLS sortant vers les services cloud.

## 9. Chronologie des mesures de référence (rappels de validité)

Réexécutable à tout moment (stack dev) :

```
bash scripts/switchover_measure.sh # contrôle bash scripts/failover_measure.sh crash # SIGKILL +  
superviseur → 3,2 s bash scripts/failover_measure.sh partition # coupure totale → 3,0 s
```

Les invariants vérifiés à chaque run : `synchronous_commit=on`, standby sync, fencing (l'ancien primary ne recapture pas), **zéro perte** (aucune transaction ackée perdue).

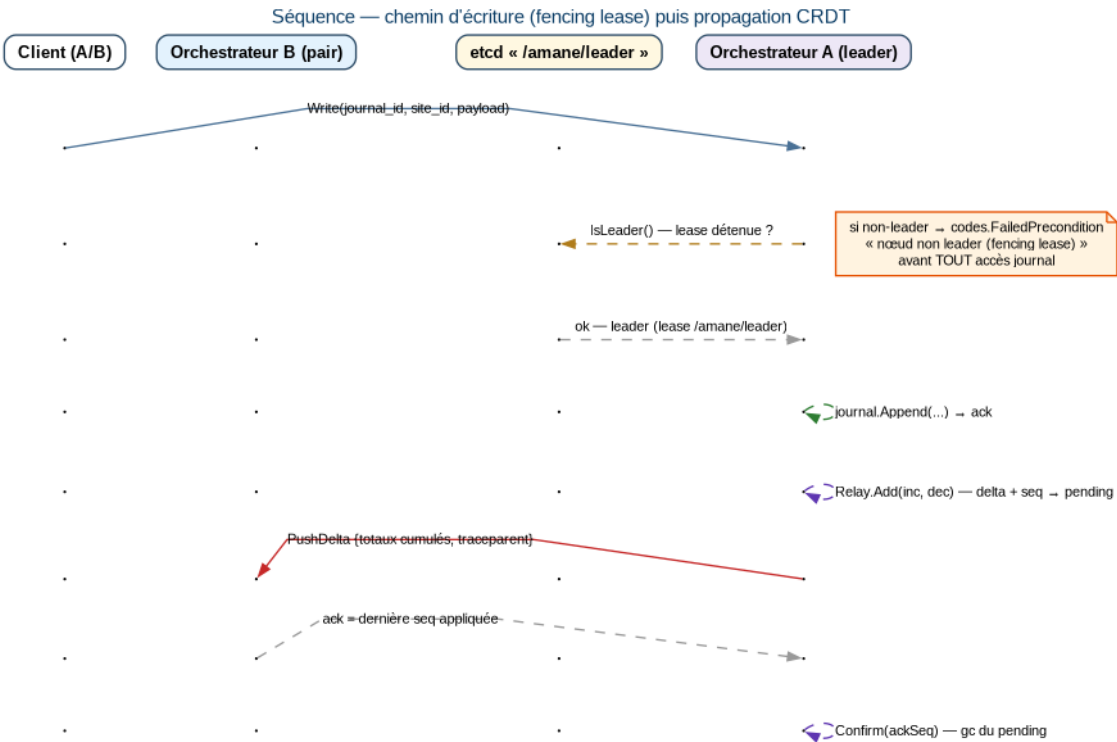
## 10. Matrice de validation

| Surface                                                | Test clé                                                | Niveau                   |
|--------------------------------------------------------|---------------------------------------------------------|--------------------------|
| Fencing Write                                          | <code>TestLeadershipFencingAgainstEtcd</code>           | etcd réel                |
| Superviseur crash                                      | <code>TestSupervisorAgainstEtcd</code> (failover < 5 s) | etcd réel + faux Patroni |
| Garde anti-partition                                   | <code>TestSupervisorPartitionGuardAgainstEtcd</code>    | etcd réel + faux Patroni |
| CRDT convergence                                       | <code>go test ./replication/ -race</code>               | unitaire 3 nœuds         |
| Propagation e2e                                        | <code>TestPropagatorE2EOverTCP</code>                   | gRPC TCP réel            |
| Contrats $A \leftrightarrow C$ / $B \leftrightarrow C$ | <code>cd tests &amp;&amp; go test ./... -race</code>    | intégration etcd réel    |
| Trace W3C                                              | <code>go test ./telemetry/ -v</code>                    | unitaire + live grpcurl  |
| Mesh                                                   | <code>go test ./mesh/ -race</code> + live 2 nœuds       | unitaire + etcd réel     |

## 11. Diagrammes de séquence du système (Graphviz)

Les échanges du système tel que développé, dans l'ordre de leur exécution (temps de haut en bas, lifelines = composants).  
Sources : `docs/diagrams/gen_seq.py` + `docs/diagrams/seq_*.dot`.

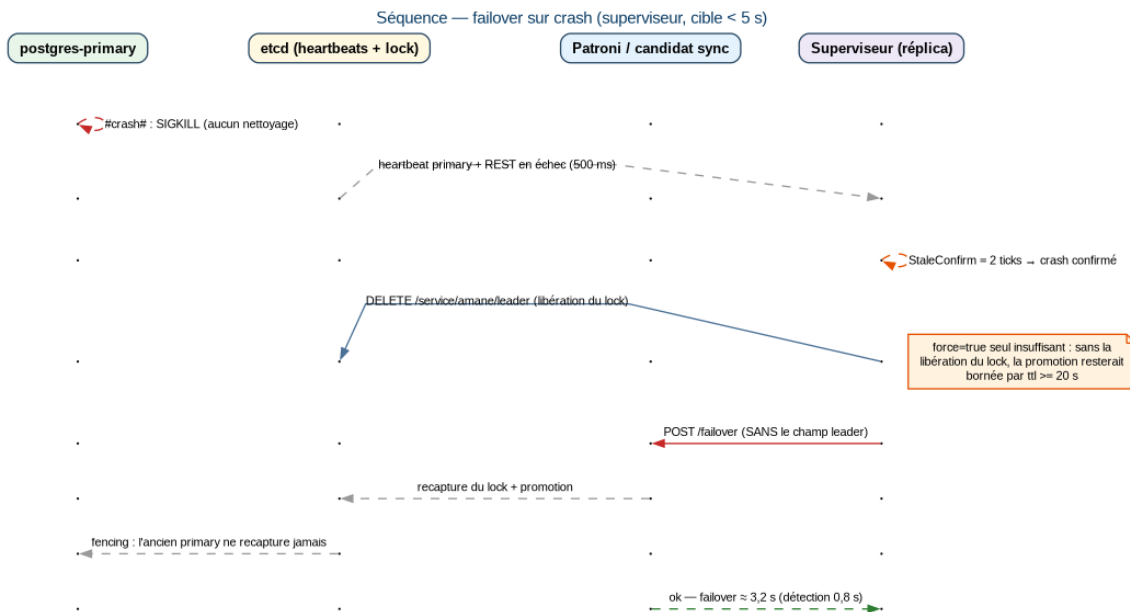
### 11.1 Chemin d'écriture — fencing lease puis propagation CRDT





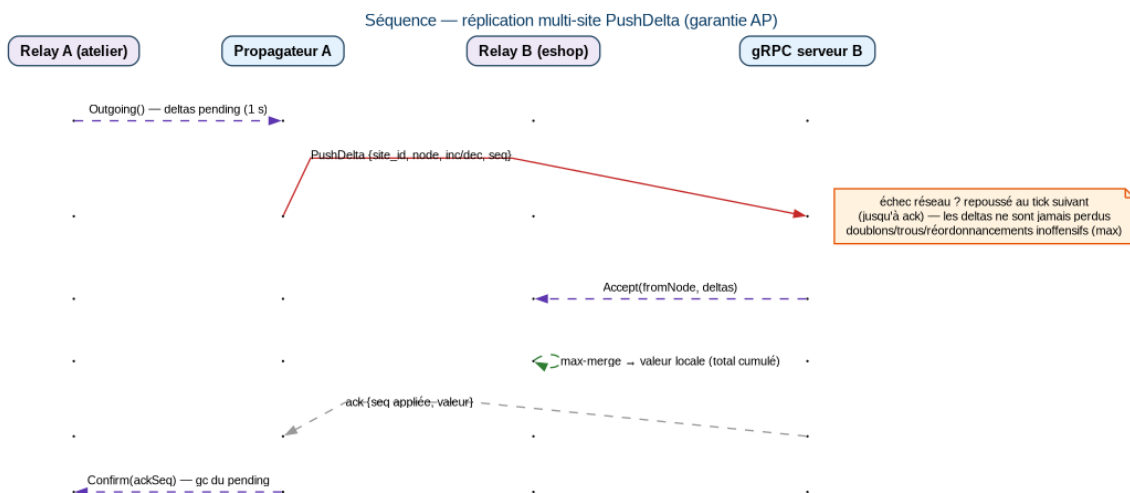
Source : docs/diagrams/seq\_write.dot. La garde `IsLeader()` passe **avant** tout accès journal ; l'échec renvoie `codes.FailedPrecondition`. Le delta (totaux cumulés) part ensuite vers le pair via `PushDelta`, l'ack vide le pending (`Confirm`).

## 11.2 Failover sur crash (superviseur, cible < 5 s)



Source : docs/diagrams/seq\_failover.dot. Détection = REST en échec + heartbeat stale (`StaleConfirm=2`) ; `DELETE` du lock Patroni puis `POST /failover` sans champ `leader` (force seul insuffisant). Fencing : l'ancien primary ne recapture jamais. ≈ 3,2 s.

## 11.3 Réplication multi-site — PushDelta / ack / Confirm



Source : docs/diagrams/seq\_push.dot. Sur échec réseau, le propagateur retente au tick suivant (deltas jamais perdus) ; le max-merge rend doublons/trous/réordonnements inoffensifs.

## 11.4 Enrôlement & révocation de machine (Interface 1, A↔C)

## Séquence — enrôlement & révocation de machine (Interface 1, Mission A ↔ C)

Machine (Mission A)

Orchestrateur C (gRPC)

etcd « /amane/members »



Source : [docs/diagrams/seq\\_enroll.dot](#). Seule la clé **publique** AK circule (règle transverse n°1) ; le quorum est recalculé à chaque enrôlement/retrait.

## 12. Organisation du repo

```

amane/ ├── proto/ # Contrat partagé proto/framework.proto (B ↔ C) – buf ├── orchestrator-go/ # Code Go
(module github.com/amane/orchestrator-go) | ├── cmd/orchestrator/ # Binaire serveur (gRPC + relay +
superviseur) | ├── cmd/wgmesh/ # Binaire mesh : publication etcd + découverte + wg0.conf | ├──
consensus/ # etcd (Raft) + lease/fencing + Patroni | ├── replication/ # relay.go (CRDT delta max-merge)
+ propagator.go (PushDelta) | ├── mesh/ # runtime WireGuard intra-site (AllowedIPs /32, keepalive 25) |
| ├── supervisor/ # Détection crash < 5 s → POST /failover (option C) | ├── grpcserver/ # Serveur gRPC
(Ping, Enroll, Write, Read, PushDelta...) | ├── telemetry/ # Trace distribuée W3C (traceparent), zéro
dépendance OTel | └── gen/amane/... # Code proto généré (buf), committé └── tests/ # Module Go ISOLÉ
github.com/amane/tests : contrats B↔C / A↔C ├── scripts/ # Mesures HA : failover_measure.sh,
switchover_measure.sh ├── docs/ # Ce document + generate_mission_c_fonctionnement_pdf.py | └──
diagrams/ # Schémas Graphviz (*.dot → PNG) + gen_seq.py (séquences) ├── docker-compose.yml # etcd +
Postgres (Spilo/Patroni) ├── buf.gen.yaml # Génération gRPC ├── tls/ # PKI DEV uniquement (clés 0600,
jamais committée) └── .opencode/skills/ # Skills mission-c-* (contextes par domaine)
    
```

Deux modules Go distincts : **orchestrator-go** (le produit) et **tests/** (le garde-fou des contrats, isolé pour ne pas être embarqué par le produit).

## 13. Comment lancer

Prérequis : Go, docker compose, etcd, **buf**, **grpcurl**, Graphviz (**dot**).

### 13.1 Stack locale (etcd + Patroni/Postgres Spilo)

```

docker compose -f docker-compose.yml up -d docker compose ps # etcd:2379, patroni:8008/8009 docker
compose down -v # wipe volumes (données de dev) curl -s localhost:8008/patroni # primary / replica
    
```

### 13.2 Serveur orchestrateur

```

go build -o /tmp/orch ./cmd/orchestrator AMANE_NODE_ID=orch /tmp/orch \ -tls-cert tls/server.crt
-tls-key tls/server.key -tls-ca tls/ca.crt # (TLS 1.3 / mTLS : openssl s_client -tls1_2 → doit ÉCHOUER)
    
```

Témoign manuel : grpcurl -cacert tls/ca.crt -cert tls/client.crt -key tls/client.key -d '{}'  
127.0.0.1:50051 amane.framework.v1.AmaneService/Ping.

### 13.3 Superviseur de failover (option C)

```
AMANE_NODE_ID=orch /tmp/orch -supervisor -patroni-scope amane \ -patroni-nodes  
postgres-primary@http://localhost:8008,postgres-replica@http://localhost:8009
```

### 13.4 Mesh WireGuard intra-site (runtime)

```
go build -o /tmp/wgmesh ./cmd/wgmesh /tmp/wgmesh -etcd localhost:2379 -name node-1 -site A -index 1 \  
-pubkey <clé publique> -privkey-file /path/to/wg_private.key -endpoint 192.168.1.20:51820 \ -conf  
/tmp/wg0-node1.conf -interval 1s # clé privée 0600 locale, jamais dans etcd ; appliquer via wg-quick  
up/down
```

### 13.5 Mesures HA (jalon 4)

```
bash scripts/switchover_measure.sh # contrôlé : ~2,2 s bash scripts/failover_measure.sh crash # SIGKILL  
: ~3,2 s (superviseur actif) bash scripts/failover_measure.sh partition # coupure totale : ~3,0 s  
(superviseur actif)
```

### 13.6 Docs

```
for f in docs/diagrams/*.dot; do dot -Tpng "$f" -o "${f%.dot}.png"; done .venv/bin/python  
docs/diagrams/gen_seq.py .venv/bin/python docs/generate_mission_c_fonctionnement_pdf.py
```

## 14. Tests : organisation et niveaux

Trois niveaux, du plus local au plus intégré :

| Niveau                              | Où                                       | etcd requis ?  | Commandes                                                                                     |
|-------------------------------------|------------------------------------------|----------------|-----------------------------------------------------------------------------------------------|
| Unitaire                            | dans chaque paquet de<br>orchestrator-go | non (fake/mem) | cd orchestrator-go &&<br>go test ./... -count=1                                               |
| Intégration etcd réel               | consensus/, supervisor/                  | oui            | AMANE_TEST_ETCD=localhost:2379<br>go test ./consensus/<br>-run<br>TestLeadershipFencing<br>-v |
| E2E gRPC TCP réel                   | grpcserver/                              | non            | go test ./grpcserver/<br>-run<br>'TestPropagatorE2E' -v                                       |
| Contrats B↔C / A↔C / write<br>gated | module isolé tests/                      | oui            | cd tests &&<br>AMANE_TEST_ETCD=localhost:2379<br>go test ./... -count=1<br>-race              |

Les tests qui exigent AMANE\_TEST\_ETCD requièrent la stack dev démarrée (13.1). Sans la variable, ils sont **skippés** — les tests unitaires restent verts hors etcd.

Carte des tests clés :

| Surface       | Test                             | Ce qu'il prouve                                          |
|---------------|----------------------------------|----------------------------------------------------------|
| Fencing write | TestLeadershipFencingAgainstEtcd | ancien leader refusé après réélection (anti split-brain) |
| Superviseur   | TestSupervisorAgainstEtcd        | crash → POST /failover < 5 s                             |

|                      |                                                                                        |                                                                       |
|----------------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Garde anti-partition | <code>TestSupervisorPartitionGuardAgainst</code>                                       | <code>REST</code> down + heartbeat frais → JAMAIS de forçage          |
| CRDT                 | <code>go test ./replication/ -race</code>                                              | convergence 3 nœuds réordonnés/doublés, gc pending, réflexion ignorée |
| Propagation          | <code>TestPropagatorPushesAndConfirms,</code><br><code>TestPropagatorE2EOverTCP</code> | ack = dernière seq, retries sans perte, pending vidé                  |
| Contrats             | <code>cd tests &amp;&amp; go test ./... -race</code>                                   | contrats B↔C / A↔C / chemin d'écriture gated                          |
| TLS                  | <code>go test ./grpcserver/ -run</code><br><code>TestCredentials</code>                | mTLS + TLS 1.3 (MinVersion=1.3)                                       |
| Trace                | <code>go test ./telemetry/ -v</code>                                                   | parse/round-trip traceparent, propagation serveur↔client              |
| Mesh                 | <code>go test ./mesh/ -race</code>                                                     | wg0.conf 0600, AllowedIPs /32, rejet <code>0.0.0.0/0</code>           |

**Bonnes pratiques** : toujours `-count=1` (pas de cache trompeur), `-race` pour les paquets concurrents (replication, relay, mesh, propagateur), et relancer `go build ./...` avant les tests après tout changement de code.